DOC 02

Dossier Principal -- Partie Commune

Contexte M2L, base de donnees partagee (MCD/MLD/SQL), architecture globale.

> *"Le projet Foot5 repose sur une architecture composee de deux applications complementaires : une application web destinee aux clients et une application lourde destinee a l'administration, partageant une base de donnees commune."*

Hakim Rayane

BTS SIO SLAM -- Mars 2026

App legere : github.com/warkozz/projet-ppe-foot5-web

App lourde : github.com/warkozz/projet-ppe-exam

Base de donnees : MySQL 'foot5' -- XAMPP

Dossier Principal -- Partie Commune

Projet PPE Foot5 -- BTS SIO SLAM

Étudiant : Hakim Rayane

Date : Mars 2026

Version : 2.1

1. Contexte Global

1.1 Présentation de l'Entreprise / Organisme

Le projet Foot5 s'inscrit dans le contexte de la Maison des Ligues de Lorraine (M2L), un établissement sportif gérant plusieurs infrastructures dont des terrains de football à cinq (5v5).

Problématique initiale :

- Gestion manuelle des réservations (agenda papier, appels téléphoniques)
- Conflits de créneaux non détectés
- Manque de visibilité pour les clients
- Charge administrative élevée pour le personnel

1.2 Besoin Global

L'organisation a besoin d'un système informatisé complet couvrant deux niveaux d'utilisation :

Besoin	Description
Côté client	Permettre aux joueurs de consulter les terrains et réserver un créneau en ligne
Côté admin	Donner aux gestionnaires un outil puissant de gestion des réservations, utilisateurs et terrains
Cohérence des données	S'assurer que les deux systèmes partagent les mêmes informations en temps réel

1.3 Solution Retenue

La solution développée repose sur deux applications complémentaires qui partagent une base de données MySQL commune :

1. Application légère (web) -- accessible depuis n'importe quel navigateur, destinée aux clients
2. Application lourde (desktop) -- installée sur le poste des gestionnaires, interface native complète

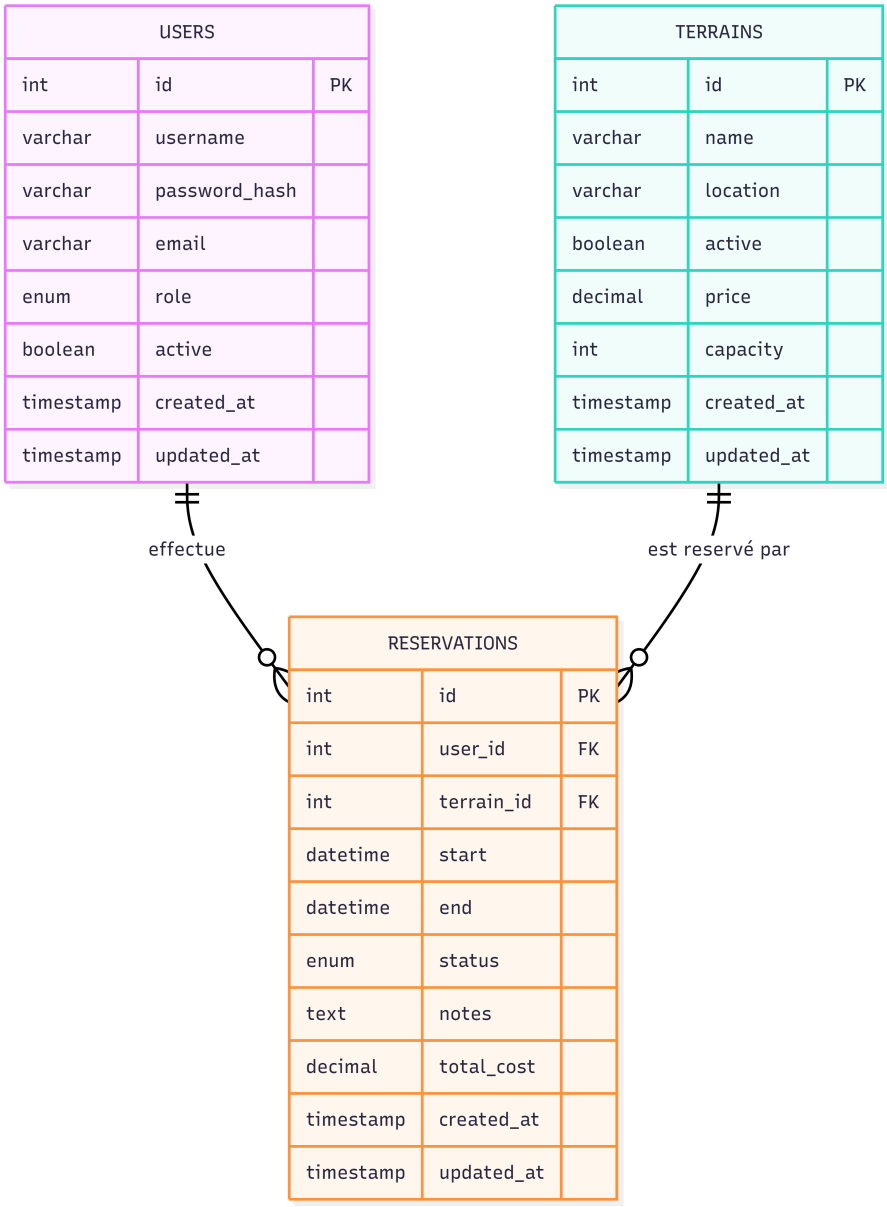
> *"Le projet Foot5 repose sur une architecture composée de deux applications complémentaires : une application web destinée aux clients et une application lourde destinée à l'administration, partageant une base de données commune."*

---

2. Base de Données Commune

> *Point clé : La base de données foot5 est partagée entre l'application web et l'application desktop. Toute modification effectuée dans l'une est immédiatement visible dans l'autre.*

2.1 Modèle Conceptuel de Données (MCD)



MCD — Modèle Conceptuel de Données Merise

Relations :

- Un UTILISATEUR peut effectuer de 0 à n RÉSERVATIONS
- Un TERRAIN peut être concerné par 0 à n RÉSERVATIONS
- Une RÉSERVATION est effectuée par exactement 1 UTILISATEUR pour exactement 1 TERRAIN

2.2 Modèle Logique de Données (MLD)

```
users (
    id              : INTEGER        PRIMARY KEY AUTO_INCREMENT,
    username        : VARCHAR(50)    UNIQUE NOT NULL,
    password_hash   : VARCHAR(255)   NOT NULL,
    email           : VARCHAR(120)   UNIQUE NOT NULL,
    role            : ENUM('user', 'admin', 'superadmin') NOT NULL DEFAULT 'user',
    active          : BOOLEAN        NOT NULL DEFAULT TRUE,
```

```
    created_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP,
    updated_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)

terrains (
    id              : INTEGER        PRIMARY KEY AUTO_INCREMENT,
    name            : VARCHAR(100)   NOT NULL,
    location        : VARCHAR(200),
    active          : BOOLEAN        DEFAULT TRUE,
    price           : DECIMAL(10,2)  DEFAULT 0.00,    ← tarif horaire (ajout mars 2026)
    capacity        : INTEGER        DEFAULT 10,       ← ex: 10 = 5v5 (ajout mars 2026)
    created_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP,
    updated_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)

reservations (
    id              : INTEGER        PRIMARY KEY AUTO_INCREMENT,
    user_id         : INTEGER        NOT NULL → FK → users(id) ON DELETE CASCADE,
    terrain_id      : INTEGER        NOT NULL → FK → terrains(id) ON DELETE CASCADE,
    start           : DATETIME       NOT NULL,
    end             : DATETIME       NOT NULL,
    status          : ENUM('pending', 'confirmed', 'cancelled') DEFAULT 'pending',
    notes           : TEXT,
    total_cost      : DECIMAL(10,2)  DEFAULT 0.00,    ← calcul auto à la création
    created_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP,
    updated_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)    REFERENCES users(id)    ON DELETE CASCADE,
    FOREIGN KEY (terrain_id) REFERENCES terrains(id) ON DELETE CASCADE,
    UNIQUE KEY unique_terrain_time (terrain_id, start, end)
)
```

2.3 Script SQL Complet

```
-- ============================================================
-- Foot5 -- Base de données commune
-- Compatible MySQL 8.0+ / MariaDB 10.5+
-- Partagée entre l'application web et l'application desktop
-- ============================================================

CREATE DATABASE IF NOT EXISTS foot5
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE foot5;

-- Table des utilisateurs
CREATE TABLE IF NOT EXISTS users (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    username      VARCHAR(50)  UNIQUE NOT NULL,
```

```
    password_hash VARCHAR(255) NOT NULL,
    email         VARCHAR(120) UNIQUE NOT NULL,
    role          ENUM('user', 'admin', 'superadmin') NOT NULL DEFAULT 'user',
    active        BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_username (username),
    INDEX idx_email    (email),
    INDEX idx_role     (role),
    INDEX idx_active   (active)
) ENGINE=InnoDB;

-- Table des terrains
CREATE TABLE IF NOT EXISTS terrains (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    location   VARCHAR(200),
    active     BOOLEAN DEFAULT TRUE,
    price      DECIMAL(10,2) DEFAULT 0.00,
    capacity   INT DEFAULT 10,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_name   (name),
    INDEX idx_active (active)
) ENGINE=InnoDB;

-- Table des réservations
CREATE TABLE IF NOT EXISTS reservations (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    user_id      INT NOT NULL,
    terrain_id   INT NOT NULL,
    start        DATETIME NOT NULL,
    end          DATETIME NOT NULL,
    status       ENUM('pending', 'confirmed', 'cancelled') DEFAULT 'pending',
    notes        TEXT,
    total_cost   DECIMAL(10,2) DEFAULT 0.00,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)    REFERENCES users(id)    ON DELETE CASCADE,
    FOREIGN KEY (terrain_id) REFERENCES terrains(id) ON DELETE CASCADE,
    INDEX idx_user_id          (user_id),
    INDEX idx_terrain_id       (terrain_id),
    INDEX idx_start_time       (start),
    INDEX idx_status           (status),
    INDEX idx_reservation_period (start, end),
    UNIQUE KEY unique_terrain_time (terrain_id, start, end)
) ENGINE=InnoDB;

-- Vue : détails des réservations
CREATE OR REPLACE VIEW reservation_details AS
SELECT
    r.id, r.start, r.end, r.status, r.notes, r.total_cost,
    u.username, u.email, u.role,
```

```
    t.name AS terrain_name, t.location AS terrain_location,
    t.active AS terrain_active, t.price AS tarif_horaire, t.capacity
FROM reservations r
JOIN users u    ON r.user_id    = u.id
JOIN terrains t ON r.terrain_id = t.id;

-- Vue : statistiques tableau de bord
CREATE OR REPLACE VIEW dashboard_stats AS
SELECT
    (SELECT COUNT(*) FROM users      WHERE active = TRUE) AS active_users,
    (SELECT COUNT(*) FROM terrains   WHERE active = TRUE) AS active_terrains,
    (SELECT COUNT(*) FROM reservations
     WHERE status IN ('pending', 'confirmed')
     AND DATE(start) = CURDATE())                        AS today_reservations,
    (SELECT COUNT(*) FROM reservations
     WHERE status IN ('pending', 'confirmed'))           AS total_active_reservations;
```

2.4 Données de Test

```
-- Données initiales (seed)
INSERT INTO terrains (name, location, active, price, capacity) VALUES
('Terrain A', 'Salle Centrale', 1, 20.00, 10),
('Terrain B', 'Salle Nord',     1, 20.00, 10),
('Terrain C', 'Salle Sud',      1, 15.00, 10),
('Terrain D', 'Salle Est',      1, 20.00, 14),
('Terrain E', 'Salle Ouest',    1, 20.00, 10),
('Terrain F', 'Salle Annexe',   0, 10.00, 10);  -- Inactif (test)

-- Comptes utilisateurs par défaut (mots de passe hachés bcrypt)
-- admin / admin123 | manager / manager123 | user1 / user123
```

2.5 Pourquoi une Base Commune ?

Avantage	Explication
Cohérence des données	Un client qui réserve en ligne est immédiatement visible dans l'app admin
Pas de synchronisation	Pas de risque de désynchronisation ou de doublons
Hachage compatible	bcrypt 12 rounds identique → même table users utilisée par les deux apps
Évolutivité	Ajout de nouvelles colonnes avec DEFAULT → rétrocompatible
Détection de conflits	La contrainte UNIQUE KEY unique_terrain_time est garantie au niveau BDD

2.6 Contraintes d'Intégrité

1. Clés primaires AUTO_INCREMENT NOT NULL sur toutes les tables
2. Clés étrangères ON DELETE CASCADE -- suppression en cascade cohérente
3. UNIQUE sur username et email -- pas de doublons utilisateurs
4. ENUM pour les rôles (user/admin/superadmin) et statuts (pending/confirmed/cancelled)
5. Contrainte temporelle CHECK (end > start) -- une réservation doit avoir une durée positive
6. Anti-conflit UNIQUE KEY unique_terrain_time (terrain_id, start, end) -- pas de double réservation

---

3. Architecture Globale

3.1 Vue d'Ensemble

```
                        +--------------------------------+
                        |        BASE DE DONNÉES         |
                        |         MySQL "foot5"          |
                        |  +--------------------------+  |
                        |  |  users                   |  |
                        |  |  terrains                |  |
                        |  |  reservations            |  |
                        |  +--------------------------+  |
                        +-----------+--------------------+
                                    |
              +--------------------+--------------------+
              |                                         |
              v                                         v
+------------------------+           +--------------------------+
```

```
|   APPLICATION WEB       |           |   APPLICATION DESKTOP      |
|   (App Légère)          |           |   (App Lourde)             |
|                         |           |                            |
|  Frontend               |           |  PySide6 (Qt6)             |
|  React + TypeScript     |           |  Interface native          |
|  Tailwind CSS           |           |  Material Design           |
|          |              |           |          |                 |
|  Backend API            |           |  SQLAlchemy                |
|  FastAPI (Python)       |           |  (accès direct BDD)        |
|  JWT Auth               |           |  bcrypt + sessions         |
+-------------------------+          +---------------------------+
       Clients/Joueurs                     Administrateurs
     (navigateur web)                    (poste de travail)
```

3.2 Flux de Données

Scénario : Un client réserve un terrain en ligne

1. Le client se connecte sur l'app web → API vérifie ses identifiants dans users
2. Il sélectionne un terrain → API récupère les données depuis terrains
3. Il choisit un créneau → API vérifie l'absence de conflits dans reservations
4. Il confirme → réservation créée dans reservations avec statut pending
5. L'admin ouvre l'app desktop → voit la réservation dans son tableau de bord
6. L'admin confirme → statut passe à confirmed dans la BDD
7. La mise à jour est immédiatement visible côté web (même BDD)

3.3 Sécurité Transversale

Mécanisme	App Web	App Desktop	Description
Hachage mdp	bcrypt (12 rounds)	bcrypt (12 rounds)	Identique, table partagée
Auth	JWT (30 min)	Session applicative	Adapté à chaque contexte
Rôles	Vérification sur API	Vérification controller	RBAC (Role-Based Access Control)
Validation	Pydantic (schémas)	SQLAlchemy + Python	Validation des entrées
Anti-duplication	Quota 2/semaine	Contrainte UNIQUE BDD	Double protection

3.4 Technologies Transversales

Technologie	Usage dans les deux apps
MySQL	Base de données unique partagée
SQLAlchemy	ORM Python utilisé côté backend web ET côté desktop
bcrypt	Hachage des mots de passe (même algorithme, mêmes rounds)
Python	Langage backend web (FastAPI) + langage desktop (PySide6)

Hakim Rayane -- BTS SIO SLAM -- PPE Foot5 -- Mars 2026

Fin du Dossier Technique

Projet PPE -- BTS SIO SLAM

Hakim Rayane -- Mars 2026

App legere (Web) : github.com/warkozz/projet-ppe-foot5-web

App lourde (Admin) : github.com/warkozz/projet-ppe-exam